

金子知適

week2

式と評価

プログラミング演習
スライド

金子

算術式

文字式

関数

論理式

条件判断

配列 (list)

式 (1) 算術式

評価結果が数 (整数 `int` または 浮動小数 `float`) になる式

評価 (計算) すると値を得る. 同じ式ならどの場所でも同じ意味.

算術演算の例

例1

$$\underbrace{(3 + 5)}_8 * 2$$
$$\underbrace{_8}_{16}$$

例2

$$- \underbrace{(-5)}_{-5}$$
$$\underbrace{_{-5}}_5$$

Q. `++5` を評価した値は?

式とは

- **原始式** —今回は— 数 e.g., 3, 5, 3.14 *primitive expression*
(評価すると自分自身になるもの)
- **式と演算子の適切な組合せ** e.g., (3+5), (1+(2+3))

(用語) 部分式: 式に含まれる式の呼称

数に対する演算子

2項演算子†	単項演算子‡
$+$ 加算	$-$ 負号
$-$ 減算	$+$ 正号♣
$*$ 乗算	(♣ 普通は使わない)
$/$ 除算	
$//$ 整数除算	
$\%$ 剰余	
$**$ べき乗	

†2項演算子



‡単項演算子

(〈演算子〉 式)

自明な場合はカッコを省略できる (詳細省略)

式と評価

プログラミング演習
スライド

金子

算術式

文字式

関数

論理式

条件判断

配列 (list)

式 (2) 文字式

評価結果が文字列になる式

評価 (計算) すると値を得る. 同じ式ならどの場所でも同じ意味.

文字演算の例

例1

$$\underbrace{('a' + 'b')}_{'ab'} + 'c'$$

$$\underbrace{\hspace{10em}}_{'abc'}$$

例2

$$\underbrace{('hail' + '2')}_{'hail2'} + 'you'$$

$$\underbrace{\hspace{10em}}_{'hail2you'}$$

式とは

- **原始式** e.g., 数, **文字(列)** (New!)
e.g., 'a', "a", 'hello', 'X', '99'
(評価すると自分自身になるもの)
- **式と演算子の適切な組合せ** e.g., 'hello'+'world'

(どちらでも良い) ' single quote, " double quote

(似ているが違う) ``` backquote

文字列に対する演算子

2項演算子 +
(式 + 式) 自明な場合はカッコを省略できる (詳細省略)

Q. 何で + が文字列の結合になるの?

..... A. Python設計者が (便利なように) 定めた

補足: 2項演算子の左右の空白は, 読みやすさのため

型

プログラミング演習
スライド

金子

算術式

文字式

関数

論理式

条件判断

配列 (list)

型 (type)

式は, 型 (かた, type) を持つ.
演算子の意味 (動作) は型毎に定義.
(雑) 数と文字列を区別する

type 演算子

Python に型を質問してみよう (確認専用, 式には使わない)

- `type(3) → int`
- `type(3+5) → int`
- `type(3.1415) → float`
- `type('abc') → str`

Q. 同じ `+` でも, `3+5` と `'3'+'5'` の結果が変わる理由は?

型が違うから. `+` 演算子は, 型に応じた計算を行う.

Q. `5-3` は 2 だけど, `'5'-'3'` は?

`TypeError`

あらかじめ定義された型についてのみ演算可能.

おまけ: `'hello!'*3` は
Pythonでは加算の略記として 定義される.
(多くの言語ではエラー)

関数

プログラミング演習
スライド

金子

算術式

文字式

関数

論理式

条件判断

配列 (list)

関数:

入出力の対応関係を表現

数学の例

$$f(x) = x^2 + 100$$

のとき

- $f(0) \rightarrow 100$
- $f(5) \rightarrow 125$

Python の例

```
def f(x):  
    return x**2 + 100
```

というセルを**実行後**

- $f(0) \rightarrow 100$
- $f(5) \rightarrow 125$

関数定義の基本パターン

```
def 関数名(x):  
    return xを使う式
```

厳密さ注意

- パターン中の `def : return` は一字一句このまま
- `return` の左に、**スペース** (空白文字) を**2つ**

拡張: 引数 (ひきすう, ~ 変数) 名は `x` でなくても良い, 複数でも良い

関数定義の基本パターン (改)

```
def 関数名(仮引数1, 仮引数2, ...):  
    return 仮引数を使う式
```

```
def add(a, b):  
    return a + b
```

式と評価と関数

プログラミング演習
スライド

金子

算術式

文字式

関数

論理式

条件判断

配列 (list)

式と評価: ここまでのまとめ

- **原始式** e.g., 数, 文字列 (評価すると自分自身になるもの)
- **式**と演算子の**適切な**組合せ
- **変数, 関数** (new!)

式を評価して**値** (あたい) を得る。

四則演算

$$\underbrace{\underbrace{(3 + 5)}_8 * 2}_{16}$$

変数 (引数) + 四則演算

$$a \leftarrow 3 \quad \underbrace{\underbrace{(\underbrace{a}_3 + 5)}_8 * 2}_{16}$$

関数+四則演算

```
def add(a, b):  
    return a+b
```

$$\underbrace{\underbrace{\underbrace{a+b}_{a=3, b=5}}_8 * 2}_{16}$$

部分式の優先順位

- 算術は常識と整合 加算 + より 乗算 * が優先。
乗算 ** はさらに優先
- 丸括弧で () 優先順位を制御。
重ねる場合も丸括弧 ((1+2)*3)**2
- 注意 **波括弧 { }** **角括弧 []** は別用途に

▷ p. 13 ▷ p. ??

変数を使う関数

関数のパターン改²

```
def 名前(仮引数1, 仮引数2...):  
    変数1 = 式  
    変数2 = 式  
    ...  
    return 式
```

- return の行の前に変数1, 2..を定義
- return する式には, 仮引数に加えて変数1, 2..を利用可能
- **厳密さ注意** def につづく行は行頭にスペースを 2[†]

[†] Pythonでは 4 が推奨だが, この授業では 2 で統一

注: しばらくの間「変数」を値に名前をつけたもの (定数) として扱う。

例題 三角形の面積 (ヘロンの公式)

3辺の長さ a, b, c に対応する三角形の面積を計算する関数

`triangle_area(a, b, c)` を作成せよ。

面積は $s = (a + b + c)/2$ として $\sqrt{s \cdot (s - a) \cdot (s - b) \cdot (s - c)}$ である。

回答例

```
def triangle_area(a, b, c):  
    s = (a + b + c) / 2.0  
    return ... # ここにs,a,b,cを使った面積の式を書く
```

式と評価 論理式

プログラミング演習
スライド

金子

算術式

文字式

関数

論理式

条件判断

配列 (list)

式 (3) 論理式

評価結果が真偽値になる式

評価 (計算) すると値を得る. 同じ式ならどの場所でも同じ意味.

論理演算の例

例1

$$\underbrace{\underbrace{(\text{True and False})}_{\text{False}} \text{ or True}}_{\text{True}}$$

例2

$$\underbrace{\underbrace{(5 < 3)}_{\text{False}} \text{ or } \underbrace{(3 < 5)}_{\text{True}}}_{\text{True}}$$

真偽値の型 (type) は **bool**

式とは

- 原始式: 数, 文字(列), **真偽値** True, False
(評価すると自分自身になるもの)
- **式**と演算子の**適切な**組合せ and, or, not, <, ==

論理演算

$\text{bool} \rightarrow \text{bool}$

(論理式 and 論理式), (論理式 or 論理式),
(not 論理式)

自明な場合はカッコを省略できる (詳細省略)

比較演算

$\text{any} \rightarrow \text{bool}$

(式 < 式), (式 > 式), (式 <= 式), (式 >= 式)
(式 == 式), (式 != 式),

等号比較は, 2文字重ねることに注意.

1文字の = は別用途に利用

補足 ブール代数

プログラミング演習
スライド

金子

算術式

文字式

関数

論理式

条件判断

配列 (list)

値

- True 真 ~ 1
- False 偽 ~ 0

真理値表 単項演算子 not

x	not x
False	True
True	False

真理値表 2項演算子 and と or

x	y	論理積	論理和
		x and y	x or y
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

日常の比喩 $x \doteq$ 商品が1000円以下, $y \doteq$ 商品がお菓子
True または False True または False

実行例

```
>>> True 1
True 2
>>> not True 3
False 4
>>> True and True 5
True 6
>>> True and False 7
False 8
>>> True or False 9
True 10
>>> (not True) or False 11
False 12
```

比較演算子と 将来の 注意点 (やや高度)

プログラミング演習
スライド

金子

算術式

文字式

関数

論理式

条件判断

配列 (list)

Python

Python の比較演算子は連鎖可能
(多項演算子, 親切)

$$\underbrace{5 > 3 > 1}_{\text{True}}$$

type 演算子で確認

- `type(True) → bool`
- `type(True+1) → int` おや?†
Q. なぜエラーにならない?

型変換

- `int('777') → 777` 便利!
- `str(777) → '777'`
- `int(True) → 1` 時に罨
- `int(False) → 0`

他の言語での注意

ほとんどの言語で,
比較演算子は**2項演算子**

失敗例

$$\underbrace{\underbrace{5 > 3}_{\text{True} \rightarrow 1} > 1}_{\text{False?!}}$$

正しい記述

2項演算子に分解して構成 (冗長)

$$\underbrace{\underbrace{5 > 3}_{\text{True}} \text{ and } \underbrace{3 > 1}_{\text{True}}}_{\text{True}}$$

† 型変換は, 暗黙に行われることがある (詳細省略)

条件判断 (2)

プログラミング演習
スライド

金子

算術式

文字式

関数

論理式

条件判断

配列 (list)

関数作成のパターン改^{2+if-elif}

```
def 名前(仮引数1, 仮引数2...):  
    変数1 = 式  
    ...  
    if <条件1 論理式>:  
        return 条件1が真の場合の式  
    elif <条件2 論理式>:  
        return 条件2が真の場合の式  
    ...  
    else:  
        return 条件が全て偽の場合の式
```

関数作成のパターン改^{2+if-nest}

```
def 名前(仮引数1, 仮引数2...):  
    変数1 = 式  
    ...  
    if <条件1 論理式>:  
        if <条件1-1 論理式>:  
            return 条件1と1-1が真のときの式  
        else:  
            return 条件1が真で1-1が偽のときの式  
    else:  
        return 条件1が偽の場合の式
```

- 分岐 (> 2): else の前に, 別の条件 elif で分岐することもできる
- 覚え方: elif $\approx$ else + if

木構造: return する代わりに, さらに if を重ねて (nest), 分岐できる

厳密さ注意

- if elif else 行末の : コロンを忘れない
- 外側の if と内側の if のインデント (行頭の空白文字数) の差が 2 であること

式と評価

プログラミング演習
スライド

金子

算術式

文字式

関数

論理式

条件判断

配列 (list)

式 (4) 配列 (list)

評価 (計算) すると値を得る. どの場所でも同じ意味.

配列の演算の例

$$\underbrace{([3] + [1, 4])}_{[3, 1, 4]} + [1, 5]$$
$$\underbrace{\hspace{10em}}_{[3, 1, 4, 1, 5]}$$

$a = [3, 1, 4, 1]$

$\underbrace{a[0]}_3$

$\underbrace{\text{len}(a)}_4$

専門的には配列とlistは別のもの

式とは

■ **原始式** e.g., 数, 文字(列), 真偽値, **配列**

e.g., $[1, 2, 3]$, $[-99]$, $[1+2, 5]$, $["hey", "yo"]$, $[]$
(式を角括弧でくくったもの. 複数の式をコンマで区切ってくくることもできる)

■ **式**と演算子の**適切な**組合せ e.g., $+$

配列に関する式

$(\text{式}_{\text{配列}} + \text{式}_{\text{配列}})$, $\text{配列}[\text{式}_{\text{整数}}]$, $\text{len}(\text{式}_{\text{配列}})$

文字列に関する式

$(\text{式}_{\text{文字列}} + \text{式}_{\text{文字列}})$, $\text{文字列}[\text{式}_{\text{整数}}]$,
 $\text{len}(\text{式}_{\text{文字列}})$